

Тема 7. Условни изрази – case, if.

Съдържание:

1. Условни изрази

1.1 Разклонения с многоклаузни функции

2. Класически конструкции за разклонение

2.1 Използване на if и unless

2.2 Използване на cond

2.3 Използване на case

2.4 Използване на специална форма

2.5 Примери за конструкции за разклонение

2.5.1 if и unless

2.5.2 case

2.5.3 cond

2.5.4 with

2.5.5 Още примери с case

2.5.6 Expressions in guard clauses

2.5.7 Още примери с cond

2.5.8 Още примери с if and unless

2.5.9 Използване на блока do/end

2.5.10 Съпоставка на конструкцията case в Erlang и Elixir

2.5.11. Съпоставка на конструкцията if в Erlang и Elixir

Тема 7. Условни изрази – case, if.

1. Условни изрази

Условните изрази в Elixir позволяват на програматора да направи решение за това какво да се случи на базата на условия. Elixir предлага различни конструкции за разклонение, за да обработва логически проверки и да извършва различни действия в зависимост от резултатите от тези проверки.

1.1 Разклонения с многоклаузни функции

В Elixir функциите могат да бъдат разширени, за да включат множество клауза, които решават различни случаи. Например, можете да използвате pattern matching в дефиницията на функция, за да опишете различно поведение на базата на аргументите.

```
defmodule Math do
  def abs(x) when x < 0, do: -x
  def abs(x), do: x
end
```

```
IO.puts(Math.abs(-5)) # Извежда: 5
IO.puts(Math.abs(3))  # Извежда: 3
```

Тема 7. Условни изрази – case, if.

2. Класически конструкции за разклонение

Elixir предлага няколко класически конструкции за разклонение, сред които if, unless, cond, и case. Те са предназначени да спомогнат за написването на ясни и четими условия.

2.1 Използване на if и unless

if позволява логическо разклонение, като проверява условие и извършва действие, ако условието е вярно.

```
x = 10
if x > 5 do
  IO.puts("x е по-голямо от 5")
end
```

unless е обратното на if. Извършва действие, когато условието е невярно.

```
unless x > 5 do
  IO.puts("x не е по-голямо от 5")
else
  IO.puts("x е по-голямо от 5")
end
```

Тема 7. Условни изрази – case, if.

2.2 Използване на cond

cond е конструкция, която позволява проверка на множество условия, подобно на if-else if в други езици.

```
age = 20
```

```
cond do
```

```
  age < 13 -> IO.puts("Детето е на възраст под 13.")
```

```
  age < 20 -> IO.puts("Тийнейджър е.")
```

```
  true -> IO.puts("Възрастен е.")
```

```
end
```

2.3 Използване на case

case се използва, за да се извършат проверки на различни стойности. Работи на принципа на моделирането (pattern matching).

```
day = :monday
```

```
case day do
```

```
  :monday -> IO.puts("Днес е понеделник.")
```

```
  :tuesday -> IO.puts("Днес е вторник.")
```

```
  _ -> IO.puts("Днес е нещо друго.")
```

```
end
```

2.4 Използване на специална форма

В Elixir има специални конструкции, като with, които дават възможност за обработка на сложни условия или последователности от операции.

```
with {:ok, value1} <- some_function(),  
    {:ok, value2} <- another_function(value1) do  
  IO.puts("И двете операции успяха с: #{value1} и #{value2}")  
else  
  {:error, reason} -> IO.puts("Настъпи грешка: #{reason}")  
end
```

2.5 Примери за конструкции за разклонение

2.5.1 if и unless

Пример с if:

```
x = 10  
result = if x > 5 do  
  "x е по-голямо от 5"  
else  
  "x не е по-голямо от 5"  
end  
IO.puts(result)
```

Тема 7. Условни изрази – case, if.

Пример с unless:

```
y = 3
unless y > 5 do
  IO.puts("у не е по-голямо от 5")
end
```

2.5.2 case

```
shape = :circle
case shape do
  :circle -> IO.puts("Фигурата е окръжност.")
  :square -> IO.puts("Фигурата е квадрат.")
  _ -> IO.puts("Неизвестна фигура.")
end
```

2.5.3 cond

```
number = 7
cond do
  number < 5 -> IO.puts("По-малко от 5")
  number < 10 -> IO.puts("По-малко от 10")
  true -> IO.puts("По-голямо или равно на 10")
end
```

Тема 7. Условни изрази – case, if.

2.5.4 with

Функции, които могат да върнат { :ok, value } или { :error, reason }

```
def func1, do: { :ok, 1 }
```

```
def func2(x), do: { :ok, x + 1 }
```

```
result =
```

```
  with { :ok, val1 } <- func1(),
```

```
    { :ok, val2 } <- func2(val1) do
```

```
    val2
```

```
  else
```

```
    { :error, reason } -> "Грешка: #{reason}"
```

```
  end
```

```
IO.puts(result) # Извежда: 2
```

2.5.5 Още примери с case

```
num = 42
```

```
case num do
```

```
  1 -> "Числото е 1"
```

```
  42 -> "Числото е отговорът на живота, вселената и всичко."
```

```
  _ -> "Някакво друго число."
```

```
end
```

2.5.6 Expressions in guard clauses

Guard clauses позволяват да добавите допълнителни условия, когато използвате case или cond.

```
x = 5
case x do
  n when n < 0 -> IO.puts("Отрицателно число")
  n when n > 0 -> IO.puts("Положително число")
  _ -> IO.puts("Нула")
end
```

2.5.7 Още примери с cond

```
value = 10
cond do
  value == 1 -> IO.puts("Едно")
  value == 10 -> IO.puts("Десет")
  true -> IO.puts("Никой от случаите не съвпада.")
end
```


2.5.8 Още примери с if и unless

```
a = true
```

```
b = false
```

```
if a do
```

```
  IO.puts("a е истина")
```

```
end
```

```
unless b do
```

```
  IO.puts("b е лъжа")
```

```
end
```

2.5.9 Използване на блока do/end

Блоковете do и end се използват за обособяване на кода, приемащ извиквания на конструкциите.

```
if true do
```

```
  IO.puts("Това ще стане")
```

```
end
```

```
unless false do
```

```
  IO.puts("Това също ще стане")
```

```
end
```

2.5.10 Съпоставка на конструкцията case в Erlang и Elixir

В Erlang case конструкцията е подобна на Elixir, но Elixir е по-лесен за четене благодарение на синтаксиса. В Elixir е подразбиращо се да имате (->), а не case ... of.

Erlang:

```
case X of  
  1 -> ok;  
  _ -> error  
end
```

Elixir:

```
case x do  
  1 -> :ok  
  _ -> :error  
end
```

2.5.11 Съпоставка на конструкцията if в Erlang и Elixir

Също както с case, Elixir предоставя по-четивен синтаксис за if, отколкото в Erlang.

Erlang:

```
if  
  X > 5 -> ok;  
  true -> error  
end
```

Elixir:

```
if x > 5 do  
  :ok  
else  
  :error  
end
```

Заклучение

В Elixir условните изрази и конструкции за разклонение предлагат разнообразие от опции за управление на потока на програмата. Те са лесни за използване и четене, което е ключово за продуктивността на програмиста. Elixir, с неговия функционален стил на програмиране, предоставя мощни конструкции за работа с условия и разклонения, които правят кода по-интуитивен.

Тема 7. Условни изрази – case, if.

Използвана литература:

- [1] SAŠA JURIC, “Elixir in Action”, Second Edition, May 2015, ISBN 9781617292019
- [2] Chris McCord, “Metaprogramming Elixir”, 2015, ISBN-13: 978-1-68050-041-7
- [3] John Paul Mueller “Functional Programming For Dummies”, 2019, ISBN 978-1-119-52751-0 (ebk); ISBN ePDF 978-1-119-52749-7 (ebk)